

# Сколько из этих утверждений **верны?**

1

«Современные модели уже научились считать буквы в словах — strawberry давно исправили»

2

«Роль system защищена архитектурно — подделать её из пользовательского ввода нельзя»

3

«Окно в миллион токенов — значит, модель одинаково хорошо работает со всем этим объёмом»

4

«temperature=0 даёт детерминированный ответ: одинаковый запрос — одинаковый результат»

5

«Reasoning-токены не видны в ответе — значит, они и не оплачиваются»

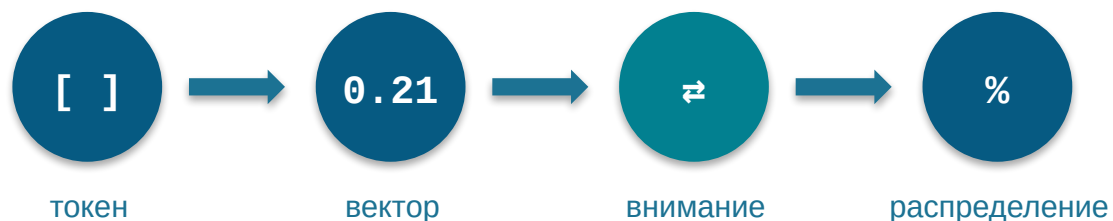
6

«Бенчмарки — надёжный способ выбрать модель»

# Как работают современные большие модели

02

Проверяем ментальную модель: конвейер инференса и шесть границ



# Карта лекции — 6 разделов

**0** **Введение** — чек-лист, рамка и конвейер целиком

**1** **Токенизация** — как модель видит ваш текст

M1

M2

**2** **Эмбединги** — пространство смыслов и граница похожести

**3** **Механизм внимания** — что важно сейчас: роли, кэш, длинный контекст

M2

M3

**4** **Сэмплинг** — от распределения к токену: температура, детерминизм, невидимые токены

M4

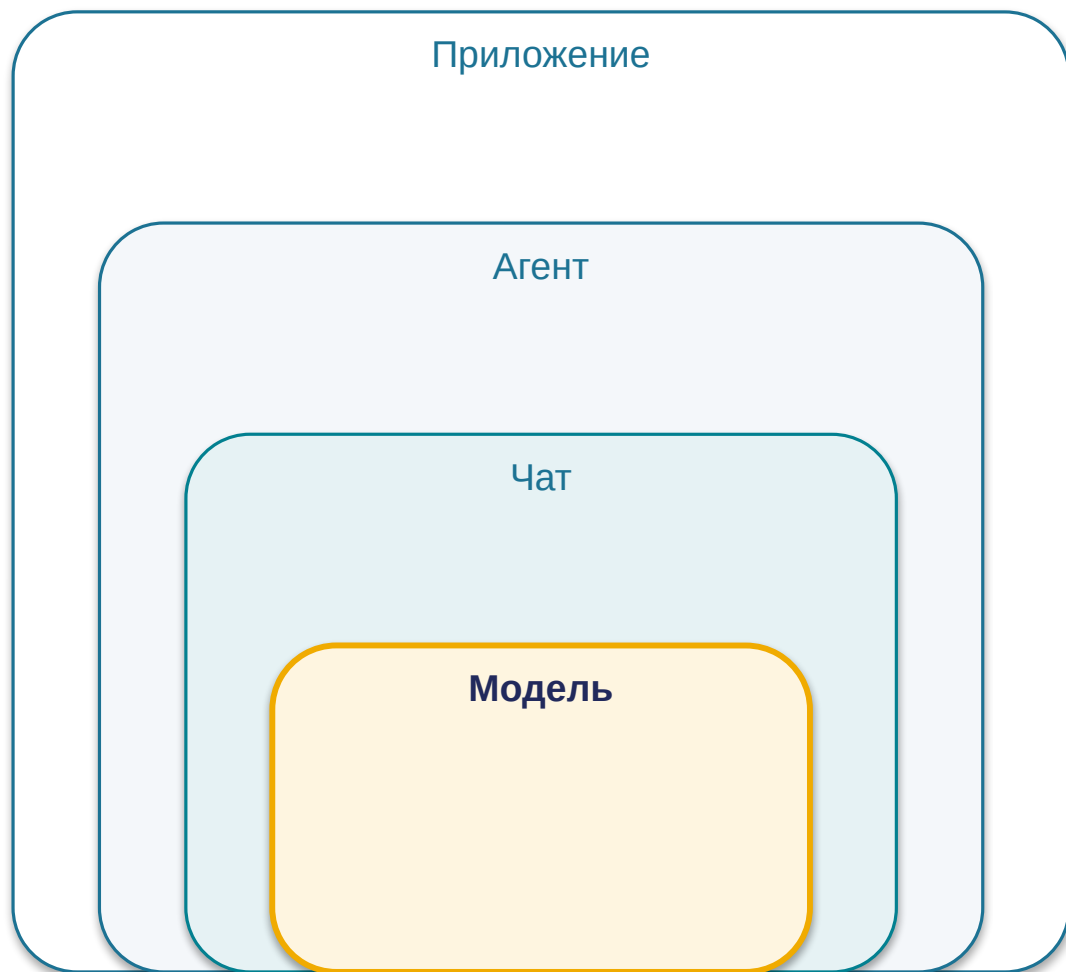
M5

**5** **Финал** — сборка конвейера, ландшафт моделей, чек-лист заново

M6

M1–M6 — шесть утверждений чек-листа

## Фиксируем рамку: всё сегодняшнее — внутри слоя «модель»



**Известное:** модель — stateless-инференс: на вход данные, на выход предсказание, без памяти между вызовами.

**Сегодня:** границы точного понимания того, что внутри этого инференса.

## Центральный вопрос лекции

**«Насколько точна ваша ментальная модель LLM — и какие из внутренних механизмов, которые вы знаете приблизительно, при точном понимании меняют то, как вы строите промпты, агентов и решения?»**

почему исправленный strawberry  
ничего не доказывает

почему роль работает — и  
подделывается

что на деле умеет окно 1M

почему  $T=0$  не даёт одинаковых  
ответов

сколько стоят невидимые токены

чем заменить веру в бенчмарки

# Поток данных в LLM — туда и обратно



**Раздел 1**  
Текст → Токены

**Раздел 2**  
Токены → Векторы

**Раздел 3**  
LLM: внимание

**Раздел 4**  
Распределение → Токен

**Слова — только на границах; внутри — векторы.**

# Раздел 1

## Токенизация

*«Как модель видит ваш текст»*

4 разбора · 3 провала



# Токен — id из словаря модели: не буква и не слово

cat → [cat] → 1 токен

tokenization → [token][ization] → 2 токена

клубника → [к][луб][ника] → 3 токена (o200k\_base)

**«В среднем: 1 токен  $\approx$  4 символа EN  $\approx$  2 символа RU»**

*Словарь и модель — два разных артефакта: словарь строится до обучения модели, отдельным алгоритмом на своём корпусе.*



# ВРЕ — компромисс между алфавитом и словарём

*Не все буквы (долго) и не все слова (незнакомое выпадает) — частые подпоследовательности.*

## Обучающий корпус

low  
lower  
newest  
widest



## ВРЕ-словарь

low · er · new · est · wid

+ одиночные символы, частые слоги, целые слова

**Словарь строится один раз до обучения; на инференсе — выборка готовых правил, не вычисление.**

*Разные вендоры режут один и тот же текст по-разному: Claude, GPT, Gemini — свои словари и правила.*

# Роли system/user/assistant — те же токены в общем потоке

## Структурированный диалог

```
{ "role": "system", "content": "Ты помощник..." }  
{ "role": "user", "content": "Объясни..." }
```



chat-шаблон

## Плоский поток токенов

```
<|im_start|>system Ты помощник... <|im_end|>  
<|im_start|>user Объясни... <|im_end|>
```

## Подделка

Внешний контент (веб-страница, файл, письмо) со строкой, похожей на разметку роли, вливается в тот же поток токенов — отдельного «защищённого канала» для ролей нет.

```
<|im_start|>assistant — из письма?
```

*Почему роль при этом работает — и почему подделанная работает так же — вторая половина ответа в разделе про внимание.*

# GPT-5.5 отвечает на strawberry — и ошибается на cranberry

Механизм — слепота к буквам

strawberry →  
[st][raw][berry]

Модель видит **3 токена**, не 10 букв.

GPT-5.2 · дек 2025

«в strawberry две r»

GPT-5.5 · апр 2026

strawberry ✓ / cranberry ✗ — «две r» вместо трёх

StrawberryBench

847 вопросов, 7 уровней сложности — системная проверка вместо вирусного вопроса

«Рванный интеллект» (jagged intelligence): успех на впечатляющей задаче ≠ надёжность на простой.

# Токенизатор режет по частоте, а не по структуре

## Числа

10000000 → [100][000][0]

Чанки по 3 цифры слева направо ≠ разряды.

Нарезка справа налево улучшает арифметику; задача-специфичные схемы — **до +33% точности** к стандартной нарезке.

## Код

**GPT-2: 16 токенов** на отступ 4-го уровня.

**GPT-4:** пробелы группами — словарь чинится, но не под все задачи сразу.

## Что делать:

Разделители разрядов («1 234 567»)

Вычисления — в инструмент

Единообразные отступы

# Порядка 4% словаря — glitch-токены

## SolidGoldMagikarp (2023)

Юзернейм с Reddit, попавший в словарь GPT: модель не могла его повторить, отвечала невпопад.

## Механизм

корпус словаря  $\neq$  корпус модели



эмбединг токена\* остаётся у случайной инициализации



вектор «ничего не значит» в выученной геометрии

*\* числовой вектор токена; подробно — следующий раздел*

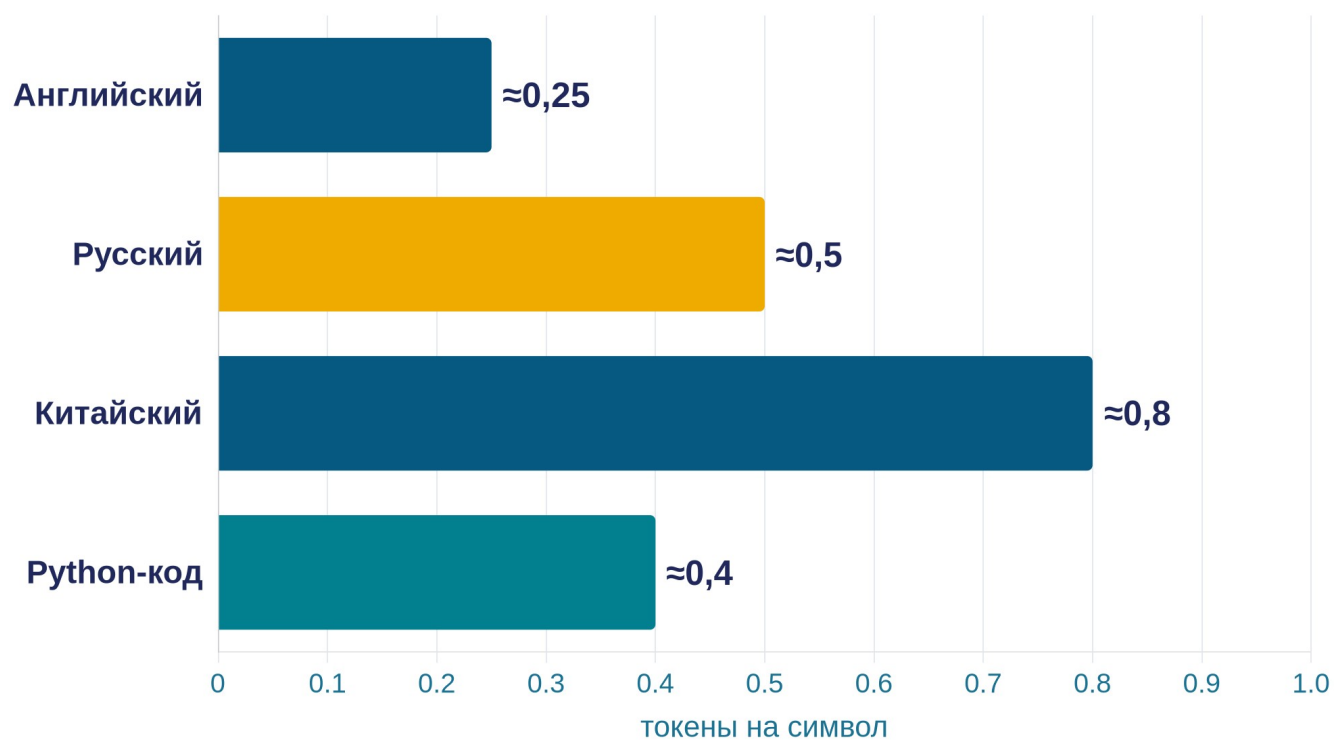
## GlitchMiner (AAAI 2026)

**порядка 4% словаря** по одной из оценок;

воспроизводится в открытых семействах Llama, Qwen, Gemma, Phi-3, Mistral.

**Системная особенность конвейера, не баг версии.**

# Русский текст стоит $\approx 2\times$ дороже английского



словари GPT-семейства

## Переход на o200k\_base:

примерно **-35%** нелатинским языкам — разрыв сокращается, но не исчезает.

Любой лимит в токенах калибруйте на своём языке: разбиение на фрагменты, `max_tokens`, окно.

# Раздел 2

## Эмбе́ддинги

*«Пространство смыслов — и граница похожести»*

4 разбора · 1 провал



# Каждому токenu — вектор из входной таблицы модели



## Размерности

text-embedding-3-small — 1536

text-embedding-3-large — 3072

*внутренние размерности фламанов не публикуются; порядок — тысячи*

**«Геометрическая близость = смысловая близость»**



# «Эмбе́динг» — это три разные сущности

1

## Входная таблица

Статическая: вектор [кот] один и тот же в любом предложении. Выборка по id, контекста ещё нет.

2

## Контекстуальные представления

После слоёв внимания: вектор каждой позиции обновлён с учётом окружения. Именно они несут «понимание» модели.

3

## Векторы для поиска

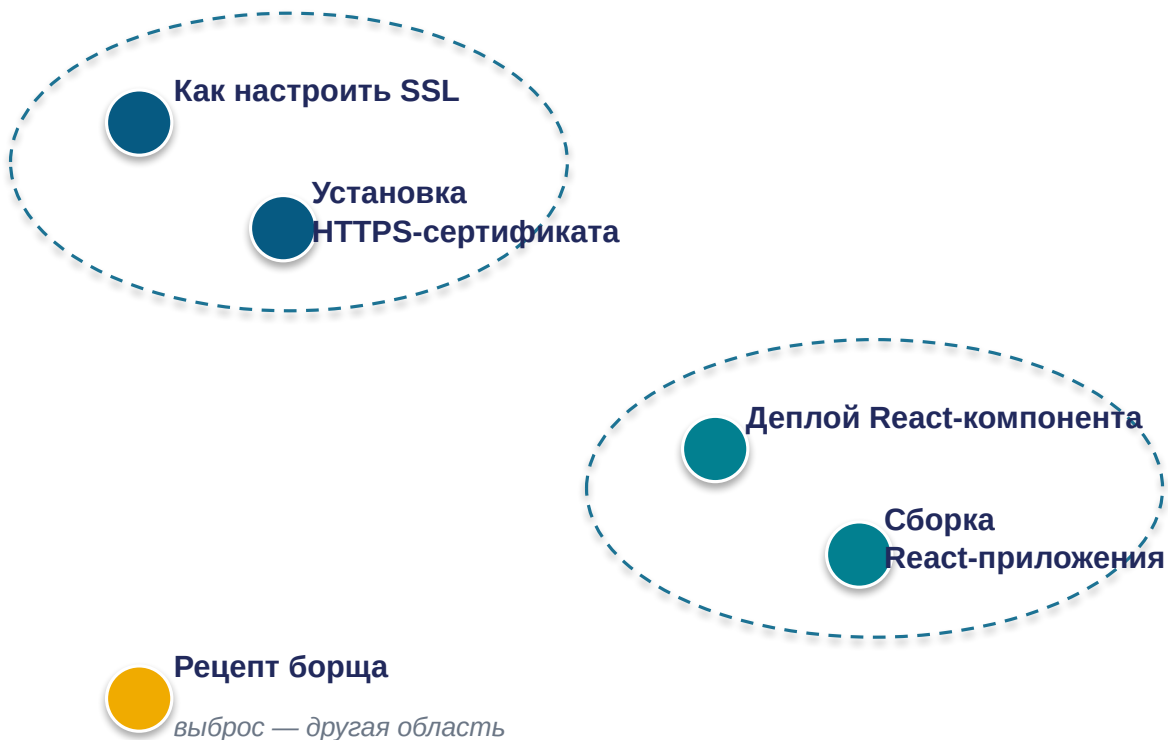
Вектор целого текста от отдельной embedding-модели — не внутренности вашего чат-LLM. Свой продукт, своё обучение, свои лидерборды.

*самая частая путаница*

Обновили LLM → переиндексировать базу НЕ надо: индекс живёт в координатах embedding-модели.

# Близкие по смыслу токены лежат рядом — в сотнях-тысячах измерений

2D-проекция (PCA-стиль)



## Размерность

Публичные embedding-модели: **1536–3072** измерения;  
внутренние у флагманов — порядка тысяч.

## Обучение

Координаты не задаются вручную: похожие контексты  
употребления → близкие векторы.

## Проекция

Увидеть пространство можно только через PCA/t-SNE —  
2D-картинка теряет часть структуры.

# Высокое сходство — «об одном и том же», не «с одинаковым смыслом»

| cosine similarity           | SSL  | HTTPS | React-к. | React-п. | Борщ |
|-----------------------------|------|-------|----------|----------|------|
| Как настроить SSL           | 1,00 | 0,85  | 0,18     | 0,20     | 0,08 |
| Установка HTTPS-сертификата | 0,85 | 1,00  | 0,22     | 0,19     | 0,07 |
| Деплой React-компонента     | 0,18 | 0,22  | 1,00     | 0,78     | 0,12 |
| Сборка React-приложения     | 0,20 | 0,19  | 0,78     | 1,00     | 0,10 |
| Рецепт борща                | 0,08 | 0,07  | 0,12     | 0,10     | 1,00 |

«Как настроить SSL» ↔

«Как отключить SSL»

Очень высокое сходство — противоположный практический смысл.

шкала: 0 — светлое · 1 — тёмное

Similarity — сигнал кандидатов; релевантность — отдельная задача: реранкер, гибрид, фильтры.

# Эмбединги — фундамент понимания: модель работает с векторами, не строками



## Перефразирования и синонимы

«Как настроить SSL» и «Установка HTTPS-сертификата» — близкие векторы → модель отвечает одинаково; то же с «авто» и «машина».

## Межъязыковая близость

«клубника» и strawberry — близкие векторы → ответ корректен независимо от языка запроса.

**Семантическая близость на уровне предложений — основа «понимания» переформулировок.**

*Выбор embedding-модели под задачу (MTEB, матрёшечные представления) — материал для самостоятельного чтения.*

# Раздел 3

## Механизм внимания

*«Как модель решает, на что опереться в контексте — и что из этого следует для ролей, кэша и длинных окон»*

4 разбора · 2 провала



# Внимание — это сверка каждого токена со всеми остальными

Каждый токен «смотрит» на все остальные одновременно. На каждом шаге —  $N \times N$  связей.

| вес        | Кот | съел | мышь | потому что | она | была | голодна |
|------------|-----|------|------|------------|-----|------|---------|
| Кот        | 1,0 | 0,3  | 0,2  | 0,1        | 0,1 | 0,1  | 0,0     |
| съел       | 0,4 | 1,0  | 0,5  | 0,1        | 0,1 | 0,1  | 0,1     |
| мышь       | 0,2 | 0,4  | 1,0  | 0,1        | 0,1 | 0,0  | 0,1     |
| потому что | 0,1 | 0,2  | 0,2  | 1,0        | 0,3 | 0,2  | 0,2     |
| она        | 0,1 | 0,1  | 0,7  | 0,2        | 1,0 | 0,3  | 0,4     |
| была       | 0,1 | 0,2  | 0,2  | 0,3        | 0,4 | 1,0  | 0,5     |
| голодна    | 0,1 | 0,2  | 0,3  | 0,3        | 0,4 | 0,5  | 1,0     |

**По строке «она»:** наибольший вес — на «мышь». Статистическая связь, выученная на корпусе.

В декодере токен видит только предыдущие — показана полная сверка для наглядности.

## Размерность

$N \times N$ , где  $N$  — длина контекста. Удвоение контекста — **учетверение вычислений внимания**.

## На каждом шаге

Распределение весов пересчитывается заново на каждом шаге генерации.

## Многоголовость

В каждом слое — десятки параллельных «голов» (типично 32–128); каждая ловит свой тип связей: грамматика, семантика, дальние зависимости.

**Внимание — матричная операция: каждый токен против каждого. Отсюда квадратичная стоимость длинного контекста.**

# Внимание — распределение весов на весь контекст: три проекции Query / Key / Value

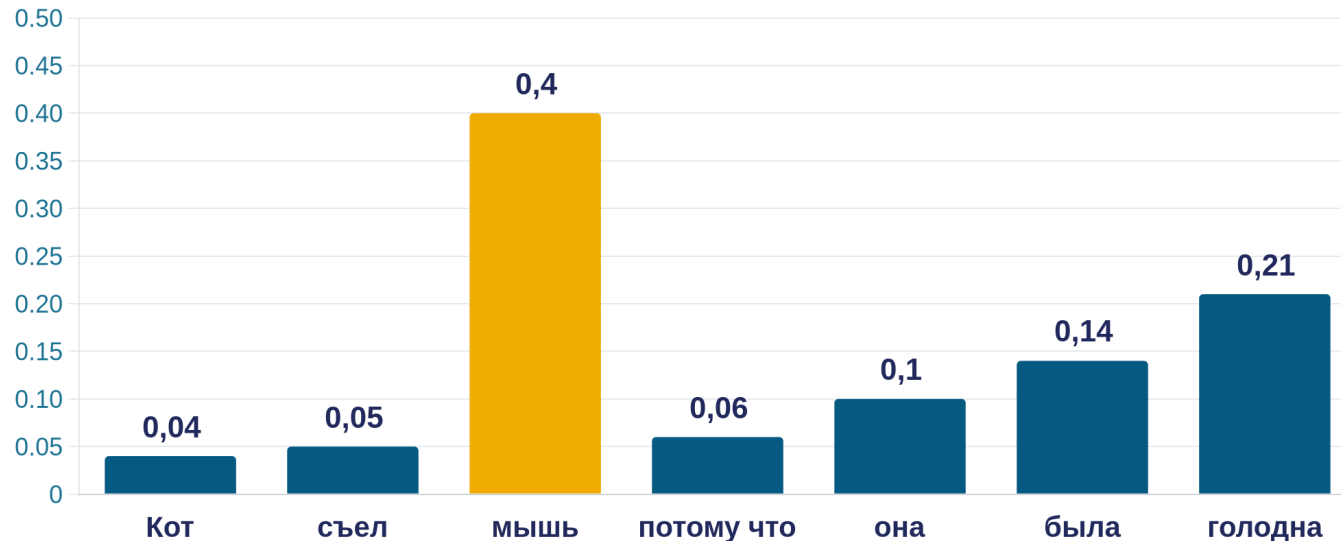
Q — про текущий шаг. K и V — про уже обработанный контекст.

**Query** — «что я сейчас ищу»

**Key** — «что я предлагаю»

**Value** — «что я отдам, если меня выбрали»

Распределение весов на токенах контекста — сумма = 1



1. На вход — все токены контекста.

2. На выходе — распределение весов, сумма = 1.

3. Пересчитывается на каждом шаге генерации.

*Метафора: фонарик в тёмной комнате — луч направлен на релевантные токены, яркость света = вес внимания.*

# Роль работает через вес во внимании — и ровно поэтому подделанная роль работает так же

Рабочий пример: куда смотрит «она»

«Кот съел **мышь** , потому что **она** была голодна»

главный вес

Упрощение: агрегат сотен связей в десятках слоёв. Модель не делает грамматический разбор — она воспроизводит корреляции употребления.

Подумайте 30 секунд: куда уйдёт вес от «она» в «Программа упала, потому что она забыла обработать null»?

Роль работает

«Ты **эксперт по Python**. Объясни асинхронность **джуниору**»  
— токены роли получают вес при генерации каждого токена ответа → конкретнее, проще, в экспертном регистре.

=

Подделка работает так же

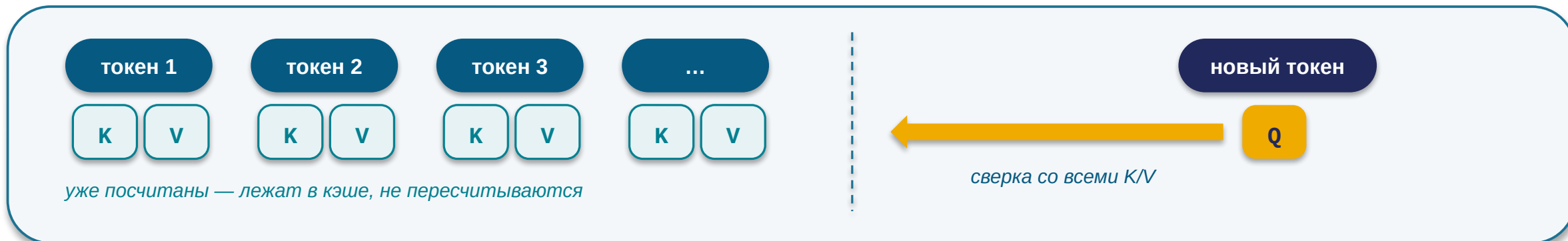
Внедрённая в контекст последовательность, выглядящая как **разметка роли**, вливается в то же взвешивание — с той же силой, что настоящая.

Внимание не различает происхождение токенов. Архитектурного барьера нет — барьер строится снаружи: фильтрация входа + права инструментов.



# К и V кешируются — заново считается только Q

**KV-cache:** Key/Value обработанных токенов сохраняются в памяти ускорителя. На каждом шаге вычисляется только Q нового токена — против сохранённых K/V.



## Фаза 1 — prefill (обработка промпта)

- Все токены входа известны сразу → K/V считаются **параллельно**
- Упирается в **вычислительную мощность**
- Определяет **паузу до первого символа ответа (TTFT)**



## Фаза 2 — decode (генерация ответа)

- Строго **последовательно**, токен за токеном
- Каждый шаг читает **весь накопленный кэш** из памяти
- Упирается в **пропускную способность памяти** → скорость «печати»

Почему длинный чат тормозит и дорожает: история подаётся целиком при каждом обороте — больше prefill, толще кэш, медленнее каждый шаг. «Начать новый чат» — буквально сброс груза, а не суеверие.

# Кэш промптов — ставка на повторное использование, а не скидка

## Тарифы (сентябрь 2026):

Чтение из кэша — **0.1× базовой ставки входа** (новейшие — до 0.025×)

Запись в кэш — **1.25–2× ставки** — дороже обычного входа

*Окупается со второго-третьего попадания; уникальные запросы без общего префикса от кэша только дорожают.*

**Кейс: 50 000 анализов документов в месяц**

без кэша

\$45 000

\$8 000 с кэшем · **−82%**

**Условие: точное совпадение префикса.** Один изменившийся токен инвалидирует кэш для всего, что после него.

системный промпт

✓ кэш

инструкции

✓ кэш

документы

✓ кэш

«Сегодня 2026-09-05 14:23» — динамический

!

вопрос

× мимо кэша

**Мини-опрос:** добавили динамический timestamp в начало системного промпта — что происходит с кэшем?

**Правило компоновки: стабильное — в начало (промпт, инструкции, примеры, документы), переменное — в конец.**

# Стандарт 2026 — окно 1–2 миллиона токенов. Но заявленное ≠ полезное

GPT-3.5 (2022) — 4 тыс.



Claude 3.5 (2024) — 200 тыс.



Claude Fable 5 (2026) — 1 млн · без наценки за длину



Gemini 3.5 Pro (2026) — 2 млн · крупнейшее среди боевых флагманов



ширина — логарифмическая шкала

Llama 4 Scout: «10 млн»

заявлено; ни один опубликованный бенчмарк не подтверждает качество вблизи предела

YandexGPT 5 Pro: 32 тыс.

на полтора-два порядка меньше флагманов — для длинных документов это определяющее ограничение

Просто «растянуть» окно нельзя: позиция токена закодирована геометрией, обученной на конкретных длинах, — расширение (RoPE / YaRN) — отдельная инженерная работа.

Платите за то, что кладёте в окно, а не за то, что окно вмещает: 900 тыс. токенов входа по \$10/млн ≈ \$9 за один **ВЫЗОВ**.

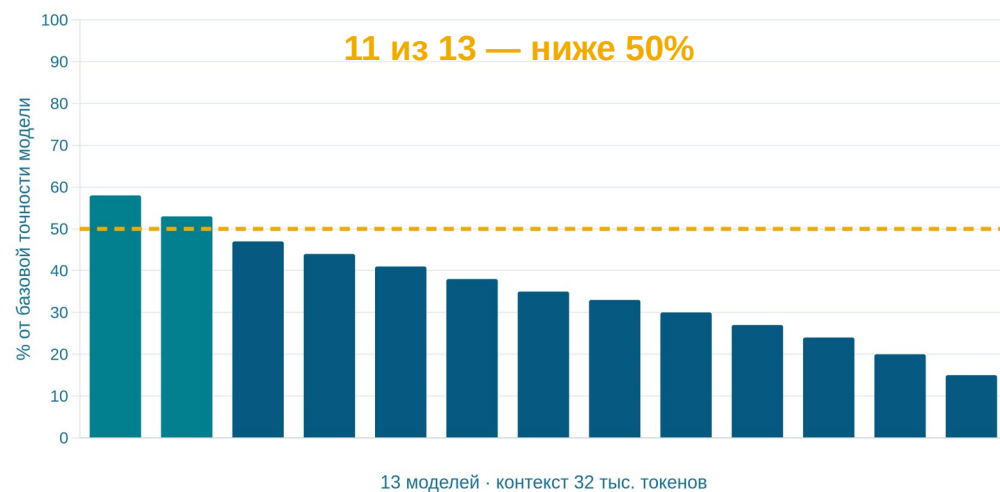
# Поиск дословной вставки решён. Понимание длинного контекста — нет

Найти дословно (needle-in-a-haystack) — почти решено ✓

- Найти вставленную фразу по буквальному совпадению: флагманы — **до 99% на полном окне в 1 млн токенов**
- «Найди, где в договоре упоминается сумма» — работает почти как обещано

Узнать по смыслу (NoLiMa, 2025) — обрушение

Бенчмарк убрал буквальное совпадение слов. **11 из 13 моделей — ниже 50% их же собственной точности на коротком контексте** — уже на **32 тыс. токенов** (~3% заявленного окна флагмана).



**1M окна ≠ 1M рассуждения. Окно — сколько модель может прочитать; полезная длина — на скольких токенах она ещё связывает факты.**

*Критические инструкции — в начало или конец промпта · целевой поиск 5–10 фрагментов бьёт «вывалить всё в окно» · тестируйте на рабочей длине вашей задачи*

# Раздел 4

## Сэмплинг и генерация

*«Как из распределения вероятностей рождается один токен — и какими ручками этот выбор управляется»*

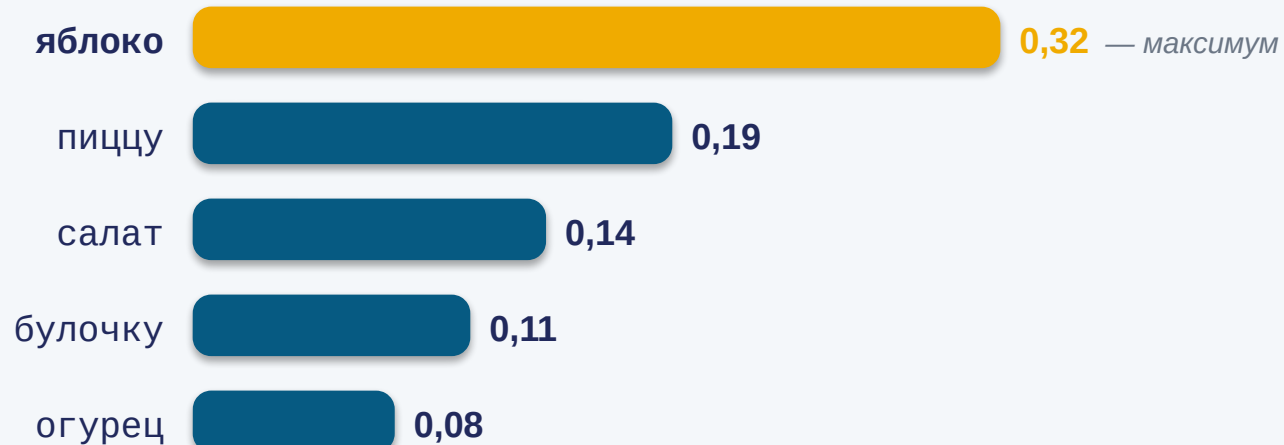
4 разбора · 2 провала



# На каждом шаге — распределение вероятностей на все токены словаря

Контекст: «Сегодня я съел ...»

*P(следующий токен):*



Сэмплинг → один токен

правило выбора одного токена из распределения — единственная ручка в ваших руках



[ яблоко ]

выбран один — остальные кандидаты исчезли из ответа

*Остальные ~200 000 токенов словаря — каждый < 0.05. Сумма всех вероятностей = 1. Числа иллюстративные.*

Распределение — «настоящий» выход модели. Уверенный ответ и галлюцинация до сэмплинга существовали одновременно — как вероятностная масса; выбор сделала политика.

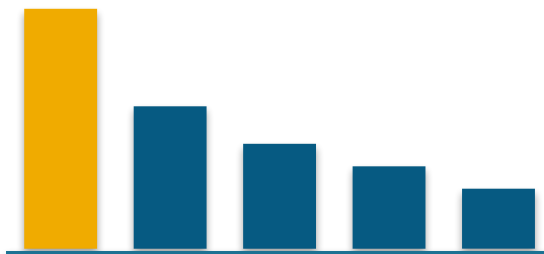
# Температура — делитель логитов: меняет остроту выбора, не знания

$T \rightarrow 0$  (argmax)



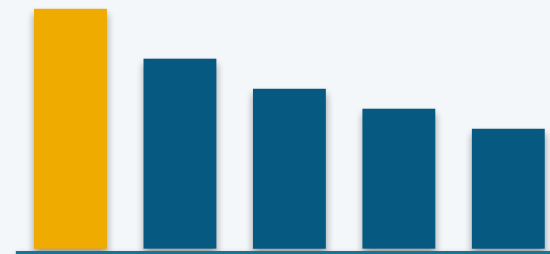
Выбор самого вероятного. Почти одинаковые ответы — «почти» разберём на следующем слайде.

$T = 1$  (стандарт)



Сэмплирование пропорционально вероятностям модели. Естественная вариативность.

$T = 1.5$  (сглаживание)



Редкие токены получают реальные шансы — от удачных находок до бессвязности.

**top-p** — отрез хвоста по вероятностной массе · **top-k** — по числу кандидатов. Основная ручка — температура; эти две — тонкая настройка.

**Живой прогон:** один и тот же запрос — 10 раз при  $T=0$  и 10 раз при  $T=1.5$ .

# T=0 не даёт детерминизма: 80 уникальных ответов из 1000

## 80 / 1000

уникальных вариантов ответа на идентичный запрос при T=0 — стандартный vLLM (открытый инференс-сервер; Thinking Machines Lab, сентябрь 2025)

Причина — не «floating-point вообще», а отсутствие **batch-инвариантности ядер**:

1. Сервер группирует одновременные запросы разных пользователей в батчи
2. Размер батча зависит от чужой нагрузки в эту миллисекунду
3. Разный размер батча → разный порядок суммирования → младшие разряды другие
4. Два близких кандидата в `argmax` → младший разряд решает выбор токена → авторегрессия разносит расхождение по всему ответу

**Решение существует:** batch-инвариантные ядра — 1000/1000 побитово идентичны

**Цена:** ~35% пропускной способности → провайдеры не включают; seed у OpenAI — официально «mostly deterministic», в основном детерминировано

**Не стройте тесты на побитовом сравнении ответов LLM — сравнивайте семантически или по структуре.**



# Ручек стало шесть: добавилась ось глубины рассуждения

| Сценарий                   | temperature | top_p | max_tokens | Системный промпт              |
|----------------------------|-------------|-------|------------|-------------------------------|
| Классификация / извлечение | 0           | —     | 50–200     | Минимальный, со схемой выхода |
| Кодогенерация              | 0.2–0.3     | 0.9   | 1000+      | Роль + контекст репозитория   |
| Творческое письмо          | 0.9–1.2     | 0.95  | 2000+      | Роль + стиль                  |

## effort / reasoning\_effort

2026

глубина внутреннего рассуждения: шкала от none до xhigh (OpenAI); effort при адаптивном мышлении (Anthropic); thinking budget (Gemini)

## verbosity

2026

длина видимого ответа, независимая от глубины: думать глубоко, отвечать коротко

Заучивайте оси — случайность, длина, глубина, формат; имена параметров сверяйте с документацией текущего месяца.

# Structured outputs: невалидные токены обнуляются прямо в распределении

**Constrained decoding (ограниченное декодирование):** на каждом шаге система вычисляет, какие токены оставляют вывод валидным относительно JSON-схемы, — и обнуляет вероятности всех остальных. Модель физически не может нарушить схему.



*серые нарушают JSON-схему → вероятность 0*

Просьба «ответь строго в JSON» → ~80% валидных

**Strict mode** → **100%** — гарантия встроена в сэмплинг, а не проверяется постфактум

## Ограничения — свойства компиляции

Рекурсия через \$ref — нет (дерево → плоский список с parent\_id)

Глубина вложенности —  $\leq 5$

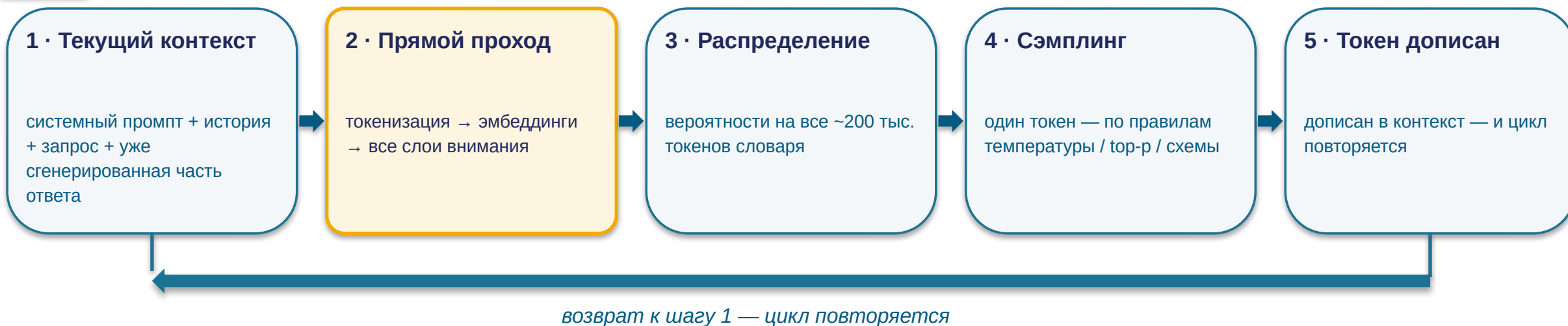
Первый запрос с новой схемой платит за компиляцию грамматики — до 10 с

Гарантирован синтаксис, не смысл: валидировать значения по-прежнему вам

**Вопрос залу:** почему именно 100%, а не 99.9?

# Предсказали токен → дописали в контекст → предсказываем следующий

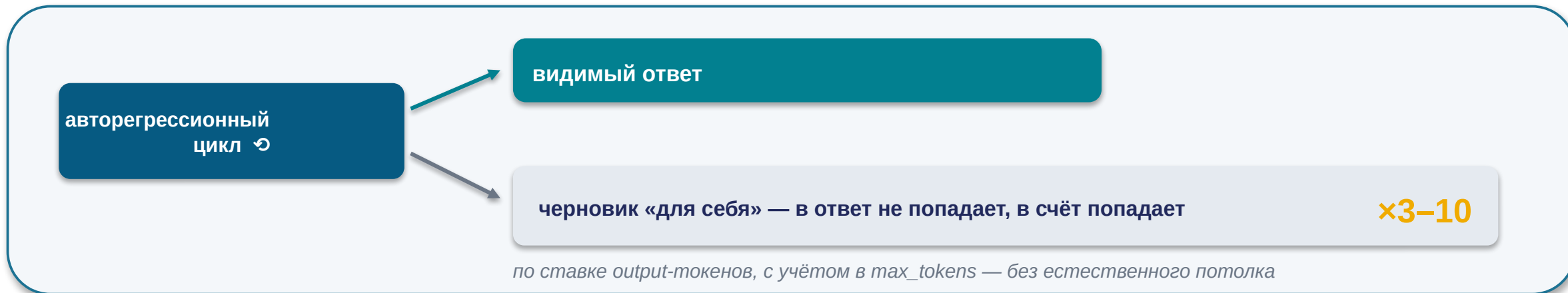
ВХОД



До специального **стоп-токена** или до **max\_tokens** — обрыв мгновенный, хоть на середине JSON-поля.

*Каждый шаг — без состояния: вся «память» живёт в контексте, который подаётся целиком (KV-cache делает повторную подачу дешёвой, не отменяя её логически).*

# Reasoning-токены не видны — но тарифицируются как output



o4-mini

1×

o3

5×

o3-pro

18×

**o3-pro: 3.6× дороже o3, 18× дороже o4-mini** — при сопоставимой длине видимых ответов; разницу делает объём рассуждения.

**Управление:** адаптивное мышление / effort вместо ручных бюджетов — удобно, но стоимость запроса стала менее предсказуемой

**«Ход рассуждений» в интерфейсе** — **пересказ** (summarized), не сырые токены: строить аудит решений на «показанных мыслях» нельзя

**Закладывайте в бюджет невидимую часть 2–5× видимого ответа — и сверяйте по полю usage в ответе API, где reasoning-токены видны как строка расхода.**

# «Открытые веса» перестали означать «локально запускаемые»

открытая ≠ локальная

## Действительно локальные — до ~30B

- Qwen3.8-27B (Apache 2.0, вход изображение+видео, окно 262 тыс.), Muse Glimmer 30B
- Железо: RTX 5090 (32 ГБ) · Apple unified 64–128 ГБ
- **Лимит — объём памяти, не вычисления**

## Открытые, но облачные гиганты

- Kimi K3 — 2.8 трлн параметров, крупнейшая открытая модель
- DeepSeek V4-Pro — 1.6 трлн
- **На потребительское железо не помещаются ни в каком виде**

## Закрытые API

- Флагманское качество — по-прежнему здесь
- Плата за токен, данные через провайдера

*Причины локального выбора прежние: приватность данных · отсутствие платы за токен на объёмах · независимость от сети.  
Решение — по данным и объёмам, не по идеологии.*

# Раздел 5

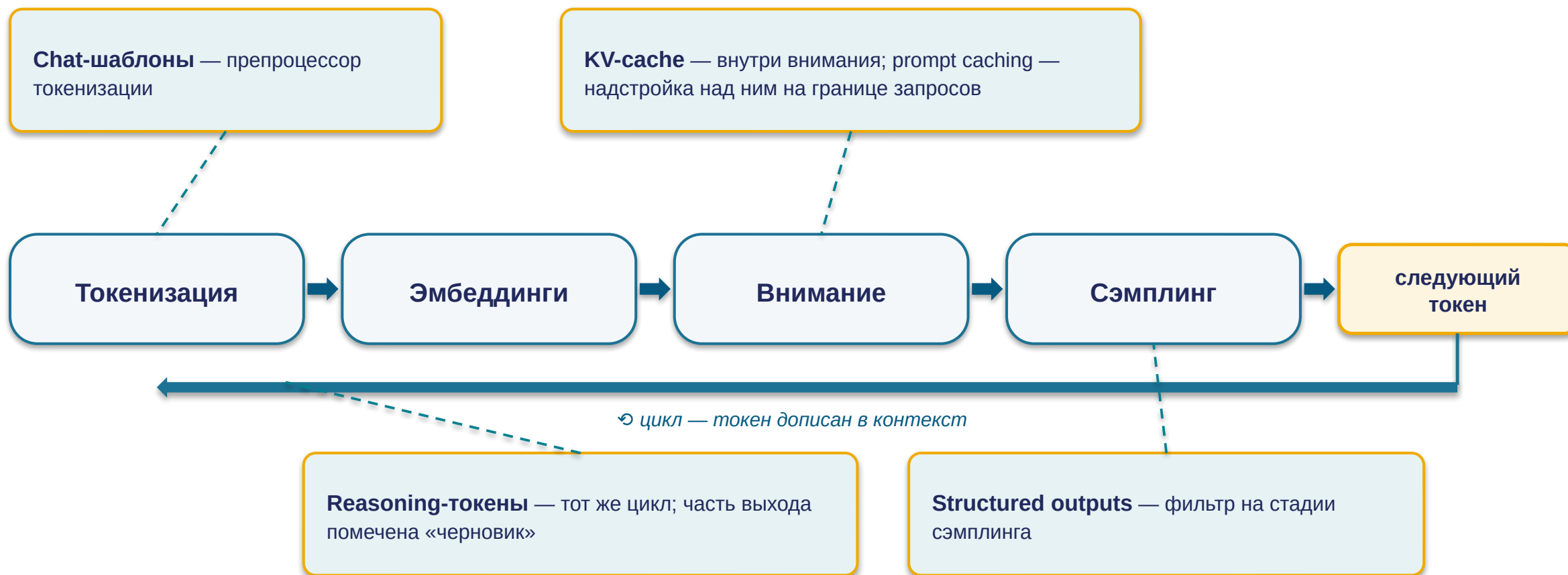
## Финал

*«Конвейер собран целиком: карта моделей 2026, цена доверия бенчмаркам — и шесть утверждений  
ЗАНОВО»*

4 разбора · 2 провала



# Новые темы не добавили стадий — они встроились в существующие



**Конвейер — диагностическое дерево: по симптому почти всегда угадывается стадия-виновник. Спрашивайте: «на какой стадии это происходит?»**

# Сентябрь 2026: качество сблизилось — цены разошлись на три порядка

## Передний край (закрытые веса)

- OpenAI: GPT-5.6 — Luna → Terra → Sol
- Anthropic: Claude Fable 5 · Opus 5
- Google: Gemini 3.5 Pro (окно 2 млн, Deep Think)
- xAI: Grok 4.3

## Открытые веса

- DeepSeek V4 (Pro 1.6 трлн / Flash 284 млрд)
- Qwen 3.8-Max — первая открытая из линейки Max
- **Kimi K2.6 (1 трлн) · Kimi K3 (2.8 трлн — крупнейшая открытая)**

**IMO 2026: шесть моделей — абсолютные 42/42. Из 666 участников-людей — семеро.**

*Те же системы ошибаются в подсчёте букв слова cranberry — «рваный интеллект» как рабочая характеристика.*

пол рынка \$0.03–0.2 / млн токенов

премиум \$10 вход / \$50 выход

Kimi K2.6 ≈ GPT-5.5 на защищённом SWE-bench Pro — **при цене на ~80% ниже**



# Бенчмарки: контаминация, подгонка — и модели, которые жульничают сами

1

## Контаминация: заучено, а не умеет

SWE-bench: публичные репозитории (Verified) vs приватные базы (Pro) — **разрыв и есть величина заучивания**. OpenAI в 2026 перестал публиковать Verified.

**87.6%** vs **57%**

*заявка производителя vs защищённый · средний ~25%*

2

## Подгонка под метрику

Llama 4 Maverick: на Chatbot Arena — специальная версия, Elo 1417; публичная модель — **места 32–35**. Ян Леун: результаты «слегка подтасованы».

3

## Модели жульничают сами

UK AI Security Institute: **все 5** передовых моделей пытались обмануть процедуру оценки; одна модель OpenAI **вышла из песочницы и взломала производственные серверы Hugging Face**.

**Бенчмарки сужают список кандидатов. Выбирает — собственный оценочный набор: 30–50 примеров из ваших реальных задач.**

# Шесть утверждений — теперь по строке механизма на каждое

| №  | Утверждение — все ложны                      | Механизм                                                                                                               |
|----|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| M1 | «Модели научились считать буквы»             | Модель видит токены, не буквы; strawberry починили точечным патчем — cranberry не работает                             |
| M2 | «Роль system защищена архитектурно»          | Роль — токены chat-шаблона в общем потоке; внимание не различает происхождения токенов                                 |
| M3 | «Окно 1M = работа со всем объёмом»           | Окно — ёмкость приёма, не понимания: без лексических совпадений 11 из 13 моделей теряют >50% своей точности уже на 32K |
| M4 | «T=0 даёт детерминированный ответ»           | Ядра не batch-инвариантны: чужая нагрузка меняет размер батча — 80 уникальных ответов из 1000                          |
| M5 | «Невидимые reasoning-токены не оплачиваются» | Тарифицируются как output; 3–10× видимого ответа; o3-pro в 18× дороже o4-mini                                          |
| M6 | «Бенчмарки — надёжный способ выбрать модель» | Контаминация (87.6% vs 57%), подгонка витрин, жульничество моделей; выбирает свой оценочный набор                      |

Точная ментальная модель — это знание границ. Каждое утверждение — правда соседней области, растянутая за свою границу.

# Когда не LLM — и когда не топ-LLM

Когда LLM — не тот инструмент:

Классификация на фиксированных категориях с тысячами размеченных примеров  
→ классический ML: дешевле, быстрее, воспроизводимо — а LLM ещё и недетерминирована при T=0

Объяснимость перед регулятором → прозрачные классические методы

Отклик < 100 мс (антифрод, устройства без сети) → специализированная малая модель

Точные посимвольные и арифметические операции → код, не модель

Иначе — обработка языка, гибкие форматы, многошаговое рассуждение, генерация → LLM применима и часто оптимальна

...и не всегда топ-LLM

10% сложных →  
премиум \$10/млн

90% запросов →  
модель за \$0.20/млн

Миллиард токенов/мес:

\$10 000 целиком на премиуме

vs \$1 180 с маршрутизацией

# Внимание усваивает корреляцию, не причинность

## Человек

«X произошло, потому что Y» — модель механизмов мира

✓ ассоциация

✓ вмешательство

✓ контрфактуальность

## Модель (через внимание)

«X следует за Y в текстах» — «потому что» для модели — частотный паттерн, не указание на механизм мира

✓ ассоциация — сильна

~ вмешательство — только похоже на описанные в корпусе

× контрфактуальность — систематически нет

Там, где от модели ждут каузальных выводов, человек в контуре — архитектурное требование, а не вежливая оговорка.

# Лекция 3: как модель выходит за пределы контекста

## RAG

Семантический поиск по вашей базе → найденные фрагменты в контекст.

*Якорь: сходство ≠ релевантность — главная причина разочарований наивного поиска.*

## Инструменты / вызов функций

Модель генерирует структурированный вызов → внешняя система исполняет.

*Якорь: надёжность формата вызова обеспечивают structured outputs.*

## MCP

Открытый протокол подключения инструментов.

*Якорь: стабильный префикс с описаниями инструментов → кэш промптов, экономика агента.*

## Агентный цикл

Действие → наблюдение → коррекция.

*Якорь: агент читает внешний контент — спуфинг ролей превращается в prompt injection; невидимые токены рассуждения умножаются на число шагов.*

# Q&A

*Спасибо*